PROBLEM 8: INTERMEDIATE CODE – TEMPORARY VARIABLE GROWTH ANALYSIS

1. HOMEBREW INSTALLATION

```
Press RETURN/ENTER to continue or any other key to abort:
==> /usr/bin/sudo /usr/sbin/chown -R seenu:admin /opt/homebrew
==> Downloading and installing Homebrew...
remote: Enumerating objects: 287, done.
remote: Counting objects: 100% (140/140), done.
remote: Compressing objects: 100% (57/57), done.
remote: Total 287 (delta 103), reused 90 (delta 83), pack-reused 147 (fro
==> /usr/bin/sudo /bin/mkdir -p /etc/paths.d
==> /usr/bin/sudo tee /etc/paths.d/homebrew
/opt/homebrew/bin
==> /usr/bin/sudo /usr/sbin/chown root:wheel /etc/paths.d/homebrew
==> /usr/bin/sudo /bin/chmod a+r /etc/paths.d/homebrew
==> Updating Homebrew...
Updated 2 taps (homebrew/core and homebrew/cask).
==> Installation successful!
```

This screenshot shows the successful installation of Homebrew on macOS. Homebrew was used as the package manager to install essential compiler development tools such as Flex, Bison, GCC, and Python libraries required for implementing the mini compiler project.

2. TOOL INSTALLATION

```
==> Next steps:
- Run brew help to get started
- Further documentation:
    https://docs.brew.sh

seenu@seenus-MacBook-Air ~ % brew install flex bison gcc
==> Fetching downloads for: flex, bison and gcc
✔ Bottle Manifest gcc (15.2.0_1)                    Downloaded   23.5KB/ 23.5KB
✔ Bottle Manifest flex (2.6.4_2)                    Downloaded   15.7KB/ 15.7KB
✔ Bottle Manifest bison (3.8.2)                     Downloaded   14.5KB/ 14.5KB
✔ Bottle Manifest libunistring (1.4.2)              Downloaded    7.3KB/  7.3KB
✔ Bottle Manifest gmp (6.3.0)                       Downloaded   13.3KB/ 13.3KB
✔ Bottle Manifest gettext (1.0)                     Downloaded   13.7KB/ 13.7KB
✔ Bottle Manifest isl (0.27)                        Downloaded   14.5KB/ 14.5KB
✔ Bottle Manifest mpfr (4.2.2)                      Downloaded   12.7KB/ 12.7KB
✔ Bottle Manifest libmpc (1.4.1)                    Downloaded   12.0KB/ 12.0KB
✔ Bottle Manifest lz4 (1.10.0)                      Downloaded   13.8KB/ 13.8KB
✔ Bottle Manifest zstd (1.5.7_1)                    Downloaded   13.2KB/ 13.2KB
✔ Bottle lz4 (1.10.0)                               Downloaded  275.2KB/275.2KB
✔ Bottle libmpc (1.4.1)                             Downloaded  157.5KB/157.5KB
✔ Bottle flex (2.6.4_2)                             Downloaded  411.7KB/411.7KB
✔ Bottle bison (3.8.2)                              Downloaded    1.1MB/  1.1MB
✔ Bottle libunistring (1.4.2)                       Downloaded    1.9MB/  1.9MB
✔ Bottle gmp (6.3.0)                                Downloaded    1.0MB/  1.0MB
✔ Bottle zstd (1.5.7_1)                             Downloaded  806.4KB/806.4KB
✔ Bottle mpfr (4.2.2)                               Downloaded    1.1MB/  1.1MB
✔ Bottle isl (0.27)                                 Downloaded    1.7MB/  1.7MB
✔ Bottle gettext (1.0)                              Downloaded   10.3MB/ 10.3MB
✔ Bottle gcc (15.2.0_1)                             Downloaded  172.9MB/172.9MB
==> Installing dependencies for flex: libunistring and gettext
==> Installing flex dependency: libunistring
==> Pouring libunistring--1.4.2.arm64_sonoma.bottle.tar.gz
🍺  /opt/homebrew/Cellar/libunistring/1.4.2: 59 files, 5.8MB
==> Installing flex dependency: gettext
==> Pouring gettext--1.0.arm64_sonoma.bottle.tar.gz
🍺  /opt/homebrew/Cellar/gettext/1.0: 2,499 files, 35.5MB
==> Installing flex
==> Pouring flex--2.6.4_2.arm64_sonoma.bottle.tar.gz
==> Caveats
```

This screenshot shows the installation process of Flex, Bison, and GCC using Homebrew. These tools provide support for lexical analysis, syntax analysis, and compilation of the compiler modules developed during the project.

3. VERSION VERIFICATION

```
seenu@seenus-MacBook-Air ~ % flex --version
flex 2.6.4 Apple(flex-35)
seenu@seenus-MacBook-Air ~ % bison --version
bison (GNU Bison) 2.3
Written by Robert Corbett and Richard Stallman.

Copyright (C) 2006 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
seenu@seenus-MacBook-Air ~ % gcc --version
Apple clang version 16.0.0 (clang-1600.0.26.6)
Target: arm64-apple-darwin23.5.0
Thread model: posix
InstalledDir: /Library/Developer/CommandLineTools/usr/bin
```

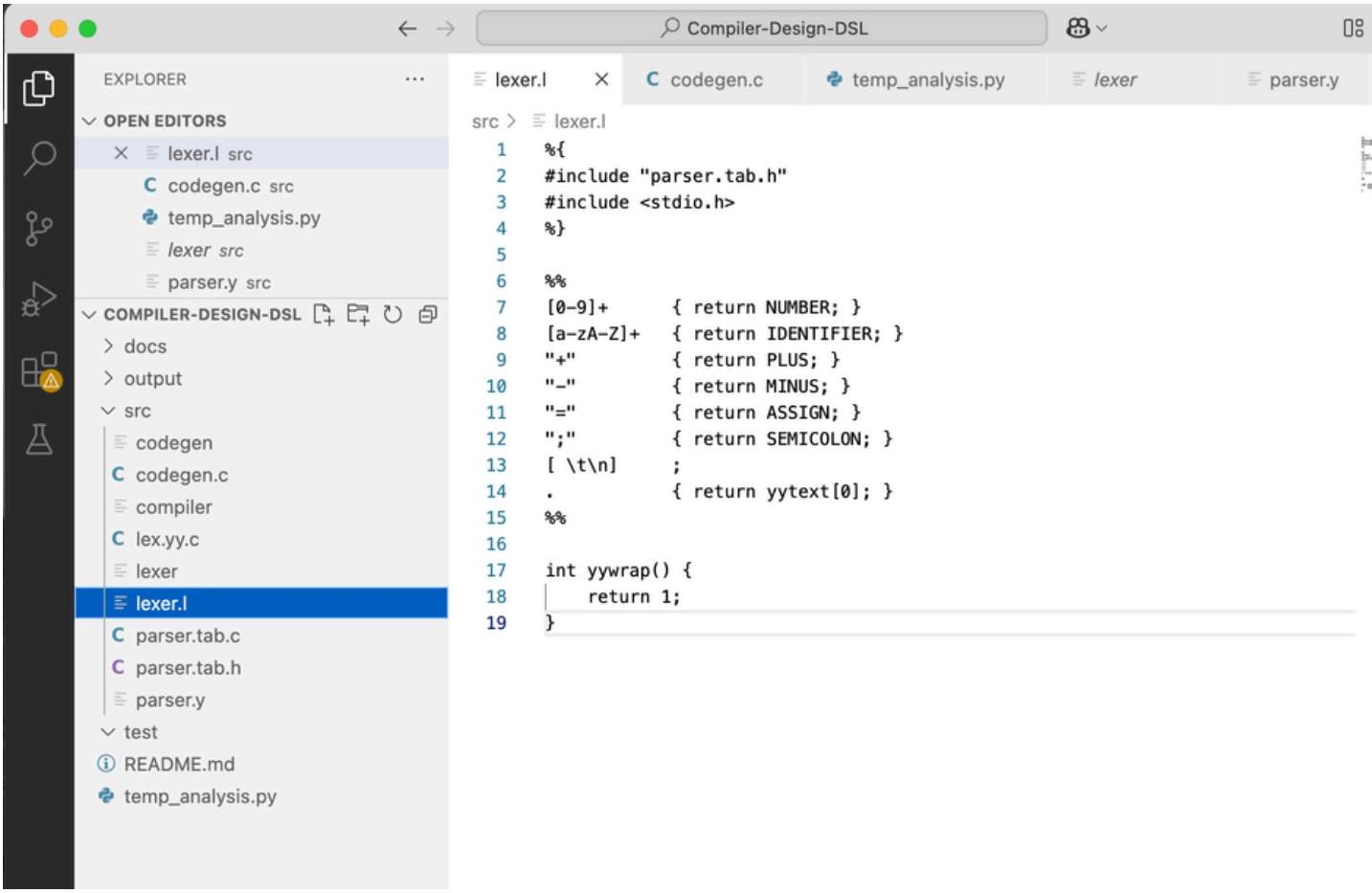
This screenshot verifies that Flex, Bison, and GCC were successfully installed and configured in the system. The displayed version information confirms that the development environment is ready for compiler implementation.

4. PROJECT FOLDER STRUCTURE

```
seenu@seenus-MacBook-Air src % cd ~/Desktop/Compiler-Design-DSL
seenu@seenus-MacBook-Air Compiler-Design-DSL % ls
README.md          docs          output          src          temp_analysis.py          test
seenu@seenus-MacBook-Air Compiler-Design-DSL %
```

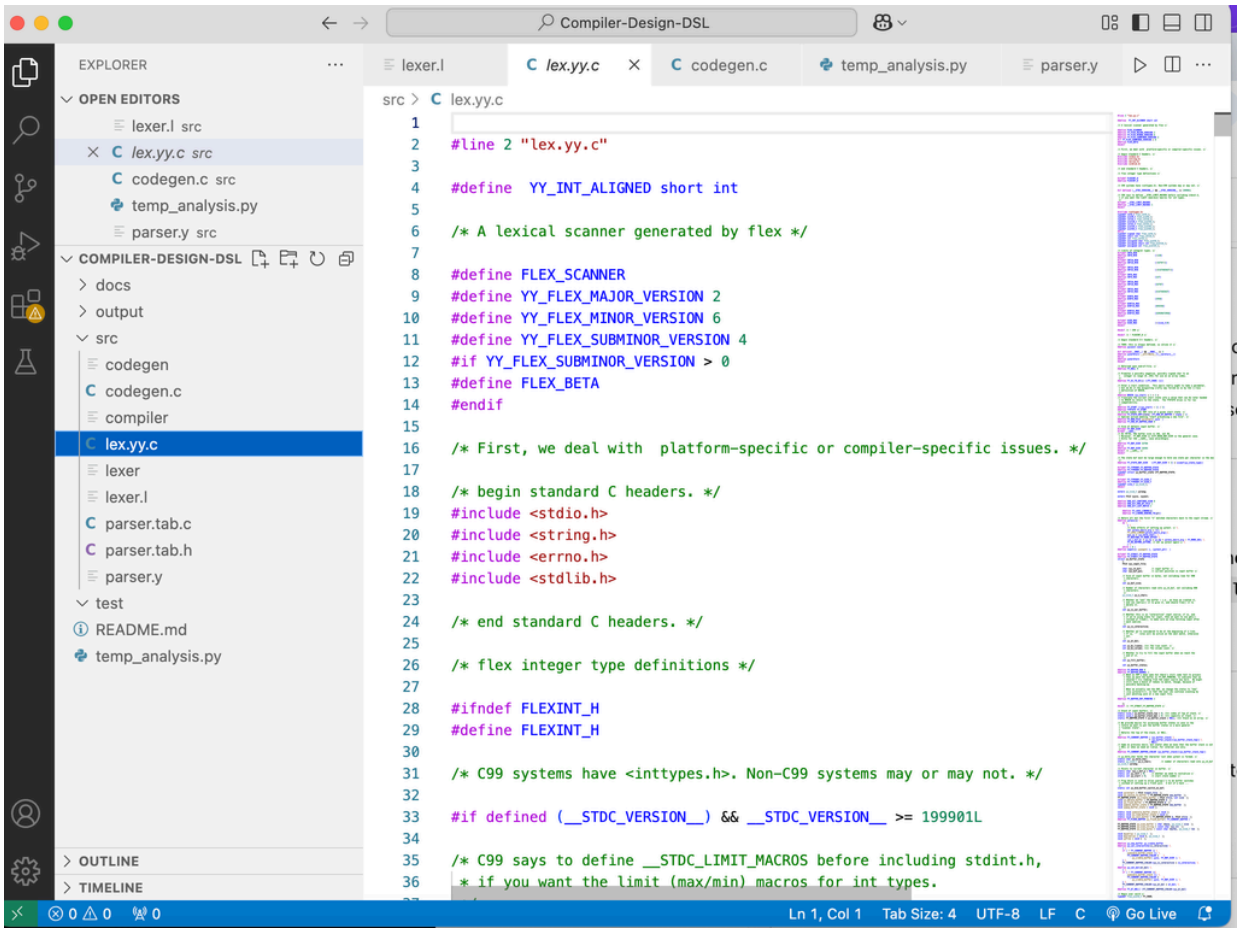
This screenshot displays the organized directory structure created for the mini compiler project. Separate folders were maintained for source files, documentation, test cases, and outputs to ensure proper project organization and maintainability.

5. LEXER.L CODE



This screenshot shows the implementation of the lexical analyzer using Flex. The lexer identifies tokens such as identifiers, numbers, arithmetic operators, assignment operators, and delimiters from the input source code.

6. FLEX EXECUTION



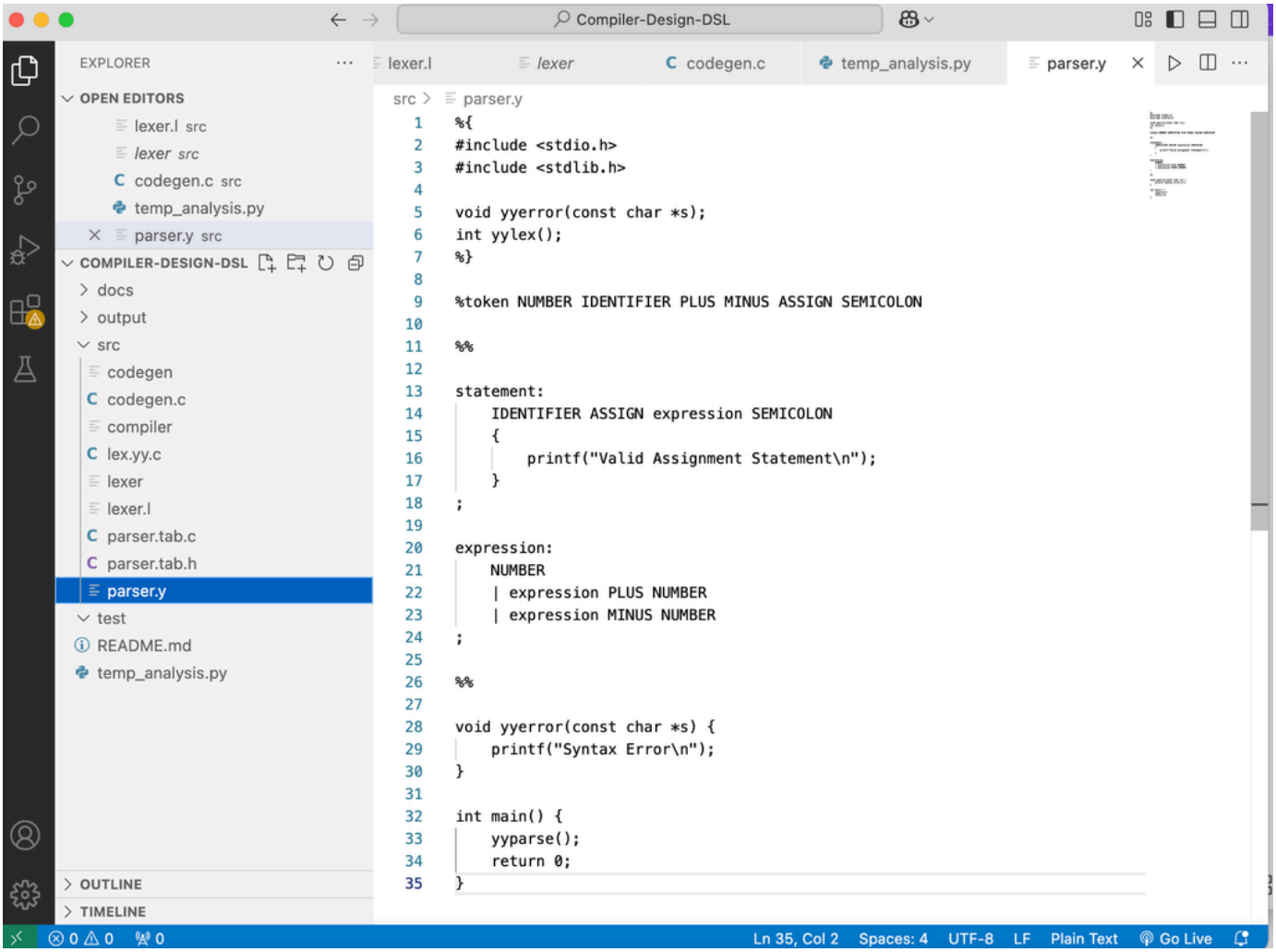
This screenshot shows the successful execution of the Flex tool to generate the lexical analyzer source file (lex.yy.c) from the lexer.l specification file.

7. LEXER COMPILATION & OUTPIUT

```
seenu@seenus-MacBook-Air src % flex lexer.l
seenu@seenus-MacBook-Air src % gcc lex.yy.c -o lexer
seenu@seenus-MacBook-Air src % ./lexer
x = 5 + 3;^D
IDENTIFIER: x
ASSIGN
NUMBER: 5
PLUS
NUMBER: 3
SEMICOLON
```

This screenshot shows the compilation of the generated lexical analyzer using GCC. The executable lexer program was successfully created without compilation errors.This screenshot shows the lexical analysis result for the input statement x = 5 + 3;. The lexer successfully tokenized the input into identifiers, operators, numbers, and delimiters.

8. PARSER.Y CODE



This screenshot shows the successful execution of Bison to generate parser source files (parser.tab.c and parser.tab.h) from the grammar specification file parser.y.This screenshot shows the successful compilation of the Flex-generated lexer and Bison-generated parser into a single executable compiler using GCC.

9. PARSER OUTPUT

```
eenu@seenus-MacBook-Air src % cd ..
eenu@seenus-MacBook-Air Compiler-Design-DSL % touch temp_analysis.py
eenu@seenus-MacBook-Air Compiler-Design-DSL % python3 temp_analysis.py
raceback (most recent call last):
 File "/Users/seenu/Desktop/Compiler-Design-DSL/temp_analysis.py", line 1, in <module>
   import pandas as pd
oduleNotFoundError: No module named 'pandas'
eenu@seenus-MacBook-Air Compiler-Design-DSL % pip3 install pandas
efaulting to user installation because normal site-packages is not writeable
ollecting pandas
 Downloading pandas-2.3.3-cp39-cp39-macosx_11_0_arm64.whl (10.8 MB)
    |████████████████████████████████| 10.8 MB 2.6 MB/s
ollecting numpy>=1.22.4
 Using cached numpy-2.0.2-cp39-cp39-macosx_14_0_arm64.whl (5.3 MB)
ollecting tzdata>=2022.7
 Using cached tzdata-2026.2-py2.py3-none-any.whl (349 kB)
ollecting python-dateutil>=2.8.2
 Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
ollecting pytz>=2020.1
 Downloading pytz-2026.2-py2.py3-none-any.whl (510 kB)
    |████████████████████████████████| 510 kB 18.3 MB/s
equirement already satisfied: six>=1.5 in /Library/Developer/CommandLineTools/Library/Frameworks/Python3.framework/Versions/3.9
nstalling collected packages: tzdata, pytz, python-dateutil, numpy, pandas
uccessfully installed numpy-2.0.2 pandas-2.3.3 python-dateutil-2.9.0.post0 pytz-2026.2 tzdata-2026.2
eenu@seenus-MacBook-Air Compiler-Design-DSL % python3 temp_analysis.py

emporary Variable Analysis

   Block  Temporary_Variables  Is_Temp_Heavy
  Block1                    1          False
eenu@seenus-MacBook-Air Compiler-Design-DSL % cd src
eenu@seenus-MacBook-Air src % ./compiler
 = 5 + 3;
alid Assignment Statement
```

This screenshot shows the syntax analysis result where the compiler validates the input assignment statement and prints “Valid Assignment Statement,” confirming correct grammar structure.This screenshot shows the implementation of the intermediate code generation module in C. The module generates three-address intermediate code using temporary variables.

10. INTERMEDIATE CODE OUTPUT

```
seenu@seenus-MacBook-Air src % touch codegen.c
seenu@seenus-MacBook-Air src % gcc codegen.c -o codegen
seenu@seenus-MacBook-Air src % ./codegen
Intermediate Code Generation

t1 = 5 + 3
x = t1
```

This screenshot shows the generated intermediate code for the arithmetic expression. Temporary variables such as t1 are created to store intermediate computation results during expression evaluation.